BonusgameWorld02
The Sample Project Mini-Cookbook

This booklet is designed to accompany the sample project `BonusgameWorld02` from the book Practical C++ Game Programming with Data Structures and Algorithms. It provides additional insights into the code logic, algorithms, and problem-solving approaches used in the implementation of this sample.

In this mini-booklet, you'll find how the sample project works and the key ideas behind its design.

Important note

If you purchased the book after **September 30, 2025**, the content of this mini-cookbook is already included in the *Optimizing rendering performance* section of *Chapter 11*, so you do not need to read this mini-cookbook separately.

Before we begin

You will need the knowledge of several chapters to completely understand how the sample works. Make sure you've finished reading till the end of *Chapter 9*, and you have checked out the sample project `BonusgameWorld01`. The best time to try out this sample is after you have played around and understood how the sample project `BonusgameWorld01` works. This booklet will not repeatedly cover topics you have learned before *Chapter 10* in the book, and the mini-cookbook of the sample project `BonusgameWorld01`.

Getting the project up and running

First, make sure you have pulled the latest updates from GitHub: https://github.com/PacktPublishing/Practical-C-Game-Programming-with-Data-Structures-and-Algorithms/tree/main.

Open the Visual Studio solution file included with this book in Visual Studio Community 2022, select the `BonusGameWorld02` project, rebuild it, and run. You should see a scene similar to *Figure 1*. You can use *WASD* to move the player character, rotate the camera with the mouse while holding the *right button*, or use the *space bar* to fire attacking magic.

Figure 1 – Running the sample project BonusGameWorld02

Optimizing rendering performance

As we continue adding more objects to a game scene, we will inevitably face significant rendering performance issues. Performance is an unavoidable challenge in game development, and the key lies in analyzing the primary causes of slowdowns and applying effective strategies to address them.

In the game industry, profiling tools (such as **RenderDoc** or **Pix**) are commonly used to identify performance bottlenecks in the code. However, you don't always need to rely on such specialized tools. Sometimes simply inserting timing measurements directly into your program, or even adjusting the number of objects being rendered, can provide enough insight to perform effective optimizations.

Below, we'll walk through an example of how to approach optimization in a game project.

Let's recall the `BonusGameWorld01` project, where we placed 600 decorative objects in the scene—far more than the earlier sample codes that demonstrated individual features. Our first step is to verify whether the sheer number of objects is the main reason for the performance drop.

The simplest way to test this is to reduce the number of objects from 600 to just 60 and then observe whether performance improves significantly. This code can be found in `TerrainEntity.cpp` (line **34**):

```
        for (int i = 0; i < 60; i++)  //change from 600 to 60
```

After saving the changes, recompiling, and running the program, we can observe that the frame rate (FPS), which reflects rendering performance, jumps significantly—from the previous 30 FPS up to 55 FPS, as shown in *Figure 2*.

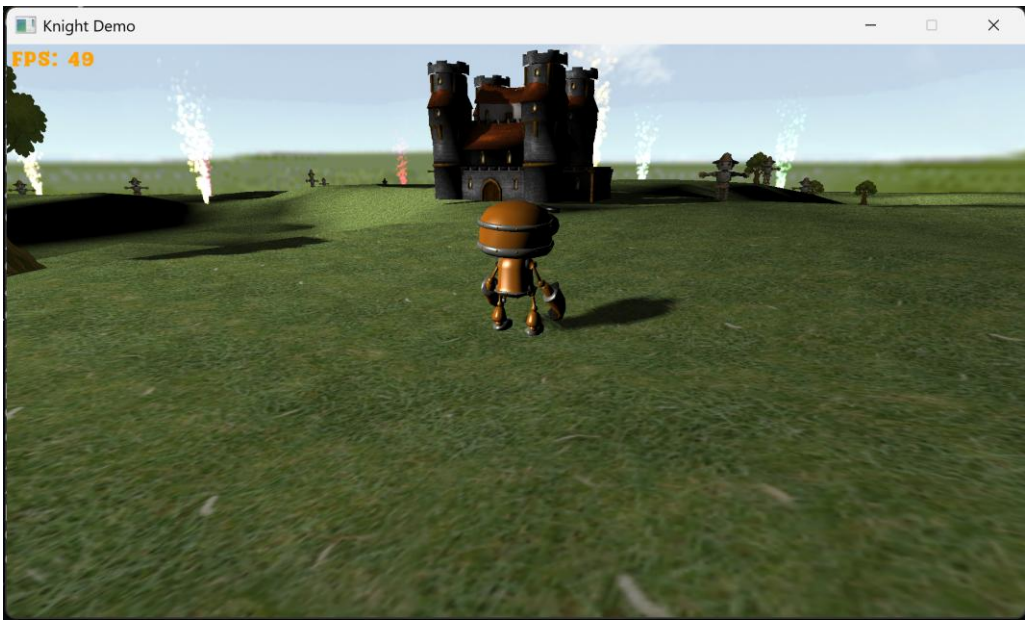

Figure 2 – The rendering performance is much better with fewer decorative trees

This is easy to understand—each decorative object also casts a shadow, meaning it must be rendered at least twice: once into the depth texture for shadow mapping, and once

onto the final screen. As a result, 600 shadow-casting objects are effectively equivalent to rendering around 1,200 objects.

However, reducing the scene to just 60 trees makes it look sparse and unconvincing. The real challenge, then, is how to maintain better performance even with 600 decorative objects in the game scene.

Two possible optimization strategies quickly come to mind:

- Render only the objects that are visible to the camera.
- Use simplified, faster rendering techniques for decorative objects that appear far away from the camera.

In the following sections, we will test both of these ideas to see how much they improve performance.

Rendering only the visible *SceneActor*

So far, all the sample projects in this book have simply rendered every `SceneActor` in the scene. This approach works fine for small demo projects and keeps the code straightforward.

However, when dealing with 600 decorative objects, we need a way to avoid sending all of them to the GPU for rendering. Instead, we should only render the objects that are visible to the camera.

We want to check whether each `SceneActor` is visible to the current active camera before calling the `Draw()` function of its attached components. If a `SceneActor` is completely outside the camera's visible range, we can simply skip it.

In fact, we already used a similar idea in *Chapter 7* when we introduced QuadTree-based terrain rendering. A terrain may consist of tens of thousands of triangles, but we only render a small portion that is visible to the camera, keeping the triangle count around 3,000 at any given time. Our implementation relied on the `SceneCamera` class's `IsBoundingBoxInFrustum()` function, using each quad tree node's bounding box to determine visibility.

We can apply the same approach here: before rendering each `SceneActor`, we check whether it falls entirely outside the camera's frustum. If it does, we skip the actor along with all its attached `Component` instances.

Although we haven't needed it so far, the `SceneActor` class defines the following member variable:

```
BoundingBox WorldBoundingBox; //in world space
```

Now comes the question: how is this `WorldBoundingBox` variable calculated?

The answer lies in the `Component` class. It contains the following member variable:

```
BoundingBox LocalBoundingBox; //in local coordinate space
```

The `LocalBoundingBox` is initialized to zero in the `Component` constructor. Each `Component` that needs to calculate and update its own bounding box can set this value inside its `Update()` function. For example, our 3D robot main character changes shape every frame due to animation. As a result, its bounding box also changes every frame and must be recalculated based on the current animation.

When the `Update()` function of a `SceneActor` is called, it will, at the end, gather the bounding regions from all of its attached `Component` instances and calculate a bounding region that encloses them. This result is then stored in the `WorldBoundingBox` variable.

If a `SceneActor` has no attached `Component`, or if none of its `Component` instances have assigned `LocalBoundingBox` any value, then the `WorldBoundingBox` will remain in its default zero-initialized state.

Since this new code will be used by every `SceneRenderPass`, it serves as a general-purpose optimization trick in 3D rendering. Therefore, we've implemented it directly in the `BuildRenderQueue()` function of `SceneRenderPass.cpp`:

```
void SceneRenderPass::BuildRenderQueue(SceneObject* pRoot) {

SceneActor* pActor = dynamic_cast<SceneActor*>(pRoot);

if (pActiveCamera != nullptr && pActor != nullptr){

  if (!(pActor->WorldBoundingBox.min.z == pActor-
>WorldBoundingBox.max.z)) {

    FrustumPlane frustumPlanes[6];

    pActiveCamera->ExtractFrustumPlanes(frustumPlanes);

    if (pActiveCamera->IsBoundingBoxInFrustum(pActor->WorldBoundingBox,
frustumPlanes) == false){

      bAddComponents = false;
```

```
        ++NumComponentsSkipped;

      }

    }

}

if (bAddComponents == true){

  //Send all components of this SceneActor

  // to GPU to draw…

}

//…

}
```

In the code above, once we confirm that the `WorldBoundingBox` of `SceneActor` is valid, we check whether the camera can see it. If the `SceneActor` lies completely outside the camera's view, it will be skipped and not rendered.

Therefore, to ensure that the `WorldBoundingBox` of a `SceneActor` is valid, each Component must define at least one or more proper `LocalBoundingBox`. Below is an example from our modified `BillboardComponent`:

```
void BillboardComponent::Update(float ElapsedTime, RenderHints* pRH) {

 __super::Update(ElapsedTime, pRH);

 SceneCamera* pSC = this->_SceneActor->GetMainCamera();

 if (pSC != nullptr){

  if (AlignType == SCREEN_ALIGNED) {

   Matrix matView = rlGetMatrixModelview();

   billUp = { matView.m1, matView.m5, matView.m9 };

  }

  LocalBoundingBox = { -size.x / 2, -size.y / 2, -0.1f, size.x / 2,
size.y / 2, 0.1f };

 }

}
```

In the original `Demo7Billboard`, the `BillboardComponent` did not include an `Update()` function. Here, we modified the `BillboardComponent` to add an `Update()` function, where the billboard's size is used to define the `LocalBoundingBox`. With this change, once the `Update()` function of `SceneActor` finishes, it will automatically call the internal `UpdateCachedWorldBoundingBox()` function to update the actor's `WorldBoundingBox`.

By adding this code to the `SceneRenderPass` base class, all classes that inherit from `SceneRenderPass` automatically gain this optimization if they use the base class's `BuildRenderQueue()` function.

For example, recall our shadow mapping algorithm: we implemented both `DepthRenderPass` and `ShadowMapRenderPass` as custom render passes, and each of them inherits from `SceneRenderPass`.

- When the `DepthRenderPass` uses the shadow-casting light as a camera to render the shadow map texture, any objects outside the shadow map's range are automatically culled.
- When the `ShadowMapRenderPass` renders the final frame, all objects that are not visible to the view camera are also skipped and not sent to the GPU.

Now compile and run the `BonusGameWorld02` project with the same 600 scene objects, resulting in a performance boost of nearly 30%!

Rendering less detail for distant objects

In addition to optimizing with camera visibility, we can also reduce the complexity of objects that are farther away from the camera. The simplest idea is that objects beyond a certain distance from the view camera (main camera) don't need to render shadows at all—we can simply skip the shadow rendering process, including the PCF shadow mapping steps that soften shadow edges using neighboring texture samples.

To achieve this, we can extend the existing `DepthRenderPass` and `ShadowMapRenderPass` classes to support distance-based optimizations that reduce rendering complexity.

Let's extend our current depth rendering pass with a new `LoDDepthRenderPass` class:

```
#include "DepthRenderPass.h"

class LoDDepthRenderPass : public DepthRenderPass {

  public:

    LoDDepthRenderPass(float cutoff, ShadowSceneLight* l);

    bool OnAddToRender(Component* pSC, SceneObject* pSO) override;

  protected:

    float CutOffDistance = 25.0f * 25.0f;

};
```

Here, we override the `OnAddToRender()` member function:

```
bool LoDDepthRenderPass::OnAddToRender(Component* pSC, SceneObject* pSO){

  float dist2 = 0;

  //If this Component do not cast shadow to other objects in the scene,
no need to redner in depth render pass

  if (pSC->castShadow == Component::eShadowCastingType:: NoShadow)

    return false;

  //Check if the object is within certain distance to the view camera

  SceneActor* pActor = dynamic_cast<SceneActor*>(pSO);

  if (pActor != nullptr){

    //Only render shadow for certain distance

    SceneCamera* pCam = pScene->GetMainCameraActor();

    //use bounding box distance if available, otherwise fallback to actor
position distance

    if (IsBoundingBoxValid(pActor->WorldBoundingBox)) {

      dist2 = PointToBoxDistanceSqr(pCam->GetPosition(), pActor-
>WorldBoundingBox);

    } else {

      //fallback to use actor position if bounding box is not valid
```

```
      dist2 =Vector3DistanceSqr(pActor->GetWorldPosition(), pCam-
>GetPosition());

    }

    if (dist2 > CutOffDistance)

      return false;

  }

  //Rest of the code

  //…

}
```

In the snippet above, it's important to note that when we refer to the camera used for distance calculations, we mean the scene's main camera.

During the depth render pass, the `pActiveCamera` in `LoDDepthRenderPass` represents the camera derived from the light's position and direction. However, when calculating the distance from the camera, we use the scene's main camera, retrieved via `GetMainCameraActor()`. If the above function `OnAddToRender()` returns false, the passed-in `Component` will not be used to render into the shadow map texture.

If the `SceneActor` has a valid `WorldBoundingBox`, we will calculate the distance between the camera's view position to this bounding box. If the `SceneActor` doesn't have a valid `WorldBoundingBox`, we then fall back to use the calculated world position of the `SceneActor` to calculate the distance.

Similarly, we need to extend `ShadowMapRenderPass` into a new class: `LoDShadowMapRenderPass`:

```
#include "ShadowMapRenderPass.h"

class LoDShadowMapRenderPass : public ShadowMapRenderPass {

  public:

    LoDShadowMapRenderPass(float cutoff, ShadowSceneLight* l, int id);

    void Render() override;

  protected:

    float CutOffDistance = 25.0f * 25.0f;

};
```

We add a new `ShouldRenderShadow()` function to determine if this `Component` in the `RenderContext` should render a shadow:

```
int LoDShadowMapRenderPass::ShouldRenderShadow(const RenderContext& rc)

{

  //No shadow receiving if the component is set to not receive shadow

  int receiveShadow = rc.pComponent->receiveShadow ? 1 : 0;

  //disable shadow receiving for far away background objects to save
performance

  if (rc.distance2 > CutOffDistance) receiveShadow = 0;

  return receiveShadow;

}
```

Unlike the `LoDDepthRenderPass` class, this time we override the `Render()` function:

```
void LoDShadowMapRenderPass::Render()

{

  //render background objects …

  //render opauqe geometry from nearest to farest

  multiset<RenderContext, CompareDistanceAscending>::iterator opaque =
pScene->_RenderQueue.Geometry.begin();

  while (opaque != pScene->_RenderQueue.Geometry.end()) {

    int receiveShadow = ShouldRenderShadow(*bk);

    SetShaderValue(shadowShader, receiveShadowLoc, &receiveShadow,
SHADER_UNIFORM_INT);

    opaque->pComponent->Draw(&Hints);

    ++opaque;

  }

  //rest of the function

  //…

}
```

This function is long, but we only need to focus on the part that renders opaque objects. Since the world position of each `Component` and its distance from the main camera have already been calculated when creating its `RenderContext`, we don't need to repeat the calculation. Instead, we can directly compare the existing `distance2` value against the `CutOffDistance`.

The `SetShaderValue()` call tells our shadow map rendering fragment shader whether this object should draw a soft shadow.

In the actual shadow rendering process of the `LoDShadowMapRenderPass` class, we already check whether each `Component` is set to receive shadows to decide if a soft shadow should be rendered. With this optimization, we simply add another condition: objects that would normally receive shadows but are beyond a distance greater than the `CutOffDistance` will be flagged as not requiring shadow rendering.

After implementing these optimizations, let's push the scene further—by increasing the number of decorative objects to 2,000—and observe the impact on rendering performance in *Figure 3*:

Figure 3 – A scene with 2000 decorative objects

Even though we increased the number of decorative objects by more than three times, performance did drop by 50%, but the cost was not in the depth rendering pass. However, if we take a closer look and measure the execution times of `Update()`, `DrawOffscreen()`, and `Draw()` functions in the `BonusGameWorld02` project, we can see that the depth rendering pass (the time spent in `DrawOffscreen()`) did not increase much. This indicates that our distance-based optimization for controlling which objects are rendered into the shadow map texture was effective.

Another benefit of limiting the range of rendering real-time shadow is that we can use a smaller orthogonal view size of the light camera. This also results in better quality of shadow map rendering.

Conclusion

We used the `BonusGameWorld02` project to demonstrate both the thought process and the implementation details involved in optimizing graphics performance.

Challenging yourself

Of course, this project still has plenty of room for further optimization, for example, avoiding repeated calculations of world coordinates and `WorldBoundingBox` values for static objects that don't move. Or, how about trying to minimize raylib's `SetShaderValue()` call to reduce unnecessary rendering status change? You might find unexpected performance boost as well!

Exploring such improvements would make excellent self-practice exercise.Happy coding and learning!